

DESIGN PATTERNS EN C & C++

Manuel pratique enrichi - explications détaillées + exemples C et C++ pour chaque pattern

Cette version enrichit le manuel précédent : les 23 patterns GoF possèdent désormais chacun un exemple, et les patterns/idiomes C/C++ supplémentaires (Object Pool, Arena, RAI, Pimpl, Reactor, Thread Pool, etc.) ont eux aussi leur mini-cas pratique. Les exemples sont volontairement courts : ils montrent la structure et l'intention sans cacher le pattern sous du code d'infrastructure.

Convention de lecture

- « C » montre une transposition idiomatique avec structures, pointeurs de fonction, handles et contexte.
- « C++ » utilise les outils naturels du langage : interfaces, templates, RAI, smart pointers, lambdas, STL.
- Un exemple n'est pas une obligation d'architecture. On choisit le pattern seulement s'il répond à un problème réel.
- Les snippets sont pédagogiques : les includes, contrôles d'erreur et détails de production sont parfois abrégés.

Comment lire chaque fiche pattern

Chaque pattern est maintenant présenté selon la même grille : intention, problème concret, mécanisme, situations d'utilisation, limites, transposition idiomatique en C, transposition en C++, puis lecture de l'exemple de code. L'objectif est de comprendre pourquoi le pattern existe avant de mémoriser sa structure.

Sommaire

1. Transposer les patterns en C et C++
2. Patterns de création - 5 GoF
3. Patterns structurels - 7 GoF
4. Patterns comportementaux - 11 GoF
5. Object Pool - chapitre détaillé
6. Mémoire et ressources : Memory Pool, Arena, Slab, RAII, Scope Guard
7. Idiomes et patterns C/C++ : Opaque Handle, Pimpl, génération, Type Erasure, CRTP, Policy-Based, Intrusive, COW, SBO, Resource Manager
8. Concurrency et systèmes : Producer-Consumer, Thread Pool, Active Object, Reactor, Proactor, Half-Sync/Half-Async, Double Buffer, Ring Buffer
9. Boussoles de choix

1. Transposer les patterns en C et C++

En C, les patterns orientés objet sont généralement matérialisés par une structure de données, un tableau de fonctions ou des pointeurs de fonction, et un pointeur de contexte jouant le rôle de this. En C++, la même intention peut être rendue par héritage dynamique, composition, lambdas, concepts ou templates. Le pattern est l'intention et la collaboration, pas le mot-clé utilisé.

C - interface par fonction + contexte

```
typedef struct {  
    void *ctx;  
    int (*authorize)(void *ctx, int cents);  
} PaymentPort;  
  
int checkout(PaymentPort p, int amount) {  
    return p.authorize(p.ctx, amount);  
}
```

C++ - interface dynamique équivalente

```
struct PaymentPort {  
    virtual ~PaymentPort() = default;  
    virtual bool authorize(int cents) = 0;  
};
```

2. Patterns de création

Factory Method

Idée en une phrase

Déléguer la décision de création d'un objet afin que le code qui l'utilise ne connaisse pas obligatoirement sa classe concrète.

Problème résolu

Le client contient des new, malloc ou switch de création dispersés. À chaque nouveau type concret, plusieurs zones doivent être modifiées, ce qui mélange logique métier et choix de construction.

Comment le pattern fonctionne

Le client travaille avec un produit abstrait. Un point de création dédié - méthode virtuelle, callback, fonction de fabrique ou fonction injectée - choisit le produit concret. La logique qui consomme le produit reste stable.

Quand l'utiliser

Utile lorsque le type concret dépend d'un format, d'une plateforme, d'une configuration ou d'un scénario qui varie, et lorsque la construction fait partie d'un point d'extension réel.

Points de vigilance

Ne remplacez pas tous les constructeurs par des factories. Si un seul type existe et que sa création est triviale, une construction directe reste souvent plus lisible.

Transposition en C

En C, une factory est souvent une fonction ou un pointeur de fonction retournant un pointeur vers une structure représentant le produit.

Transposition en C++

En C++, elle peut être une méthode virtuelle, une lambda, un std::function, une factory injectée ou une simple fonction retournant std::unique_ptr<T>.

Lecture de l'exemple

Exemple : un importeur doit choisir un Parser CSV, JSON ou XML. Le workflow d'import reste identique ; seul le point makeParser varie.

Intention

Déléguer la création d'un produit afin que le code principal ne connaisse pas le type concret.

Exemple concret

Un importeur choisit un parseur CSV ou JSON sans que le workflow d'import connaisse les classes concrètes.

Transposition

C : Passer une fonction factory ou une table create().
C++ : Méthode virtuelle de création, lambda ou std::function.

Mini-exemple C

```
typedef Parser *(*ParserFactory)(void);
void import_file(ParserFactory make) {
    Parser *p = make();
    parser_parse(p, "data");
    parser_destroy(p);
}
```

Mini-exemple C++

```
struct Importer {
    virtual std::unique_ptr<Parser> makeParser() = 0;
    void run() { auto p = makeParser(); p->parse("data"); }
};
```

Point de vigilance - Ne pas remplacer chaque new/malloc par une factory si aucun choix de création ne varie.

Abstract Factory

Idée en une phrase

Créer une famille complète d'objets compatibles sans exposer les classes concrètes de cette famille.

Problème résolu

Plusieurs objets doivent être choisis ensemble. Si le client sélectionne chaque classe séparément, il peut fabriquer des combinaisons incohérentes, par exemple un bouton "dark" avec une boîte de dialogue "light".

Comment le pattern fonctionne

Une factory abstraite expose plusieurs opérations de création. Chaque factory concrète produit tous les éléments d'une même famille. Le client sélectionne une famille puis ne manipule que les contrats abstraits.

Quand l'utiliser

Pertinent pour des thèmes UI, des backends de rendu, des familles de drivers, des environnements mémoire/SQL ou des ensembles de composants dépendant d'une plateforme.

Points de vigilance

Ajouter une nouvelle famille est généralement facile. Ajouter un nouveau type de produit est plus coûteux car toutes les factories existantes doivent évoluer.

Transposition en C

En C, on représente naturellement la factory par une structure de pointeurs de fonctions `make_xxx`.

Transposition en C++

En C++, une interface `UiFactory` avec plusieurs méthodes virtuelles fonctionne bien ; des templates ou policies peuvent éviter le dispatch dynamique si les familles sont connues à la compilation.

Lecture de l'exemple

Exemple : `UiFactory` produit `Button` et `Dialog`. `DarkUiFactory` garantit que les deux produits appartiennent au même thème.

Intention

Créer une famille cohérente d'objets liés sans exposer leurs classes concrètes.

Exemple concret

Une UI existe en thème clair et sombre : bouton, dialogue et menu doivent provenir de la même famille.

Transposition

C : Struct regroupant plusieurs fonctions `make_x()`.
C++ : Interface de factory ou policy de famille.

Mini-exemple C

```
typedef struct {
    Button *(*make_button)(void);
    Dialog *(*make_dialog)(void);
} UiFactory;

void build(const UiFactory *f) {
    Button *b = f->make_button();
    Dialog *d = f->make_dialog();
}
```

Mini-exemple C++

```
struct UiFactory {
    virtual std::unique_ptr<Button> button() = 0;
    virtual std::unique_ptr<Dialog> dialog() = 0;
};
```

Builder

Idée en une phrase

Construire progressivement un objet complexe en séparant l'accumulation des choix de la validation finale.

Problème résolu

Un constructeur possède trop de paramètres, beaucoup sont optionnels, l'ordre devient ambigu ou certaines combinaisons sont invalides. Les constructeurs télescopiques deviennent illisibles.

Comment le pattern fonctionne

Le builder stocke temporairement les options. Le client configure uniquement ce qui l'intéresse, puis `build()` vérifie les invariants et fabrique l'objet final, idéalement valide et immuable.

Quand l'utiliser

Utile pour les requêtes, configurations, objets immuables riches, protocoles possédant beaucoup d'options ou scénarios de test nécessitant des valeurs par défaut lisibles.

Points de vigilance

Pour trois paramètres simples et obligatoires, un constructeur nommé ou une fonction `make_xxx` est souvent préférable. Le builder ne doit pas devenir un moyen de contourner les invariants.

Transposition en C

En C, une structure `Builder` avec champs et flags de présence fonctionne bien ; une fonction `build` valide puis retourne ou initialise le résultat.

Transposition en C++

En C++, une API fluide peut retourner `*this` et profiter du move semantics. `build()` est un bon point pour transférer l'état vers un objet final immuable.

Lecture de l'exemple

Exemple : `Request::Builder` configure URL, timeout et TLS sans imposer un constructeur `Request(url, timeout, tls, ...)`.

Intention

Construire progressivement un objet complexe en séparant étapes de construction et objet final.

Exemple concret

Construire une requête HTTP avec URL obligatoire, headers optionnels, timeout, proxy et authentification.

Transposition

C : Struct Builder mutable puis `build()`.

C++ : API fluent, named builder, éventuellement objet final immuable.

Mini-exemple C

```
typedef struct {
    const char *url;
    int timeout_ms;
    bool tls;
} RequestBuilder;

Request build(RequestBuilder *b) {
    return (Request){ b->url, b->timeout_ms, b->tls };
}
```

Mini-exemple C++

```
auto req = Request::Builder{}
    .url("https://api")
    .timeout(1500ms)
    .tls(true)
    .build();
```

Point de vigilance - Un constructeur simple reste préférable si l'objet possède peu de paramètres non ambigus.

Prototype

Idée en une phrase

Créer un nouvel objet en copiant un objet déjà configuré plutôt qu'en reconstruisant toute sa configuration.

Problème résolu

La création normale est coûteuse ou le code ne connaît pas à l'avance le type concret à instancier. Un objet modèle contient déjà l'état initial pertinent.

Comment le pattern fonctionne

Chaque prototype définit une opération de copie. Le client demande un clone puis adapte l'état spécifique. La décision importante est la politique de copie : superficielle, profonde ou partage contrôlé.

Quand l'utiliser

Pratique pour les objets de jeu préconfigurés, arbres de scènes, configurations complexes, documents modèles ou objets dont le type concret est déterminé dynamiquement.

Points de vigilance

Le danger principal est une copie ambiguë : pointeurs partagés par erreur, ressources système dupliquées, identifiants clonés ou ownership mal défini.

Transposition en C

En C, le clone est souvent un pointeur de fonction attaché à la structure ou une fonction `clone_type` dédiée.

Transposition en C++

En C++, une méthode virtuelle `clone()` retournant `std::unique_ptr<Base>` est une forme classique et sûre pour les hiérarchies polymorphes.

Lecture de l'exemple

Exemple : un moteur de jeu conserve un prototype `Enemy` configuré ; `spawn()` clone ce prototype pour créer rapidement une nouvelle instance.

Intention

Créer un nouvel objet en copiant un objet déjà configuré.

Exemple concret

Dans un moteur de jeu, un prototype `EnemyOrc` configuré est dupliqué pour créer rapidement plusieurs ennemis similaires.

Transposition

C : Callback clone + politique de copie explicite.
C++ : Constructeur de copie ou virtual clone().

Mini-exemple C

```
typedef struct Entity Entity;
struct Entity {
    Entity *(*clone)(const Entity *self);
    int hp;
};
Entity *spawn(const Entity *prototype) {
    return prototype->clone(prototype);
}
```

Mini-exemple C++

```
struct Enemy {
    virtual ~Enemy() = default;
    virtual std::unique_ptr<Enemy> clone() const = 0;
};
```

Point de vigilance - La copie superficielle de pointeurs propriétaires peut créer des doubles free ou du partage involontaire.

Singleton

Idée en une phrase

Garantir qu'un service possède une seule instance dans un périmètre donné, tout en fournissant un moyen d'y accéder.

Problème résolu

Certaines ressources doivent réellement être uniques : registre global de process, backend de métriques, accès à un périphérique exclusif. Une création multiple serait incohérente.

Comment le pattern fonctionne

La création est contrôlée et l'instance n'est initialisée qu'une fois. Le pattern classique combine unicité et accès global, mais ces deux décisions peuvent être séparées.

Quand l'utiliser

À réserver aux cas où l'unicité est une contrainte du problème, pas simplement une commodité pour éviter de passer une dépendance.

Points de vigilance

L'accès global rend les dépendances invisibles, crée de l'état partagé et complique les tests. En C++ moderne, un objet unique géré par la composition racine est souvent préférable à un getInstance() partout.

Transposition en C

En C multithread, pthread_once ou un mécanisme équivalent permet une initialisation sûre.

Transposition en C++

En C++11+, un static local de fonction est initialisé de manière thread-safe. Mais l'injection d'une référence vers une instance unique reste souvent plus propre.

Lecture de l'exemple

Exemple : Metrics& metrics() retourne une instance unique. Pour les tests, on préférera souvent injecter un Metrics& dans les composants.

Intention

Garantir une instance unique dans un scope donné et fournir un moyen contrôlé d'y accéder.

Exemple concret

Une table de métriques process-wide doit être initialisée une seule fois.

Transposition

C : static + pthread_once/call_once selon la plateforme.
C++ : Static local thread-safe depuis C++11, ou mieux : cycle de vie géré par DI.

Mini-exemple C

```
static Metrics g_metrics;
static pthread_once_t once = PTHREAD_ONCE_INIT;
static void init_metrics(void) { metrics_init(&g_metrics); }
Metrics *metrics_instance(void) {
    pthread_once(&once, init_metrics);
    return &g_metrics;
}
```

Mini-exemple C++

```
Metrics& metrics() {
    static Metrics instance;
    return instance;
}
```

Point de vigilance - Une instance unique n'implique pas qu'un accès global statique soit souhaitable. L'injection explicite reste souvent plus testable.

3. Patterns structurels

Adapter

Idée en une phrase

Traduire une interface existante vers le contrat attendu par le client.

Problème résolu

Une librairie, un SDK ou un composant legacy fournit une API incompatible : noms, paramètres, conventions d'erreur ou représentation des données ne correspondent pas au vocabulaire du domaine.

Comment le pattern fonctionne

L'adapter implémente le contrat attendu et délègue au composant existant en réalisant les conversions nécessaires. Il concentre la traduction à la frontière.

Quand l'utiliser

Très fréquent pour les API réseau, drivers matériels, fichiers, bibliothèques C, SDK de paiement et code legacy.

Points de vigilance

Un adapter ne doit pas être une simple redirection 1:1 si cela laisse les types du fournisseur traverser la frontière. Il doit réellement protéger le modèle client.

Transposition en C

En C, une petite structure contenant ctx + fonctions adapte bien un vendor SDK à un port interne.

Transposition en C++

En C++, une classe dérivée d'une interface cible encapsule la référence ou le smart pointer vers l'adapté.

Lecture de l'exemple

Exemple : Vendor::write retourne un nombre d'octets ; Transport::send veut un booléen. L'Adapter traduit la convention et masque le Vendor au client.

Intention

Convertir l'interface d'un composant existant vers celle attendue par le client.

Exemple concret

Le domaine attend send(bytes), mais le SDK fournisseur expose vendor_write(handle, ptr, len, flags).

Transposition

C : Wrapper + fonction d'adaptation.

C++ : Classe Adapter qui implémente le port du domaine.

Mini-exemple C

```
static int vendor_send(void *ctx, const uint8_t *p, size_t n) {
    Vendor *v = ctx;
    return vendor_write(v, p, n, 0) == (int)n;
}
```

Mini-exemple C++

```
class VendorAdapter : public Transport {
    Vendor& v_;
public:
    bool send(Bytes b) override { return v_.write(b.data(), b.size(), 0); }
};
```

Bridge

Idée en une phrase

Séparer deux dimensions de variation indépendantes afin qu'elles puissent évoluer sans explosion combinatoire de sous-types.

Problème résolu

L'héritage modélise simultanément deux axes : par exemple type de notification x canal. Chaque nouvelle variante multiplie les classes nécessaires.

Comment le pattern fonctionne

L'abstraction contient une référence vers une implémentation. Une hiérarchie représente l'axe fonctionnel, l'autre l'axe d'implémentation. Les deux sont composées au runtime.

Quand l'utiliser

Utile avec rendu x plateforme, forme x API graphique, notification x transport, stockage logique x backend physique.

Points de vigilance

Si un seul axe varie réellement, la composition simple suffit ; créer deux hiérarchies sans besoin concret ajoute du coût cognitif.

Transposition en C

En C, l'axe d'implémentation est souvent un vtable + ctx stocké dans la structure principale.

Transposition en C++

En C++, l'abstraction possède une référence/pointeur vers une interface Implementor, souvent injectée au constructeur.

Lecture de l'exemple

Exemple : Notification définit le message ; Channel décide comment le transmettre par email, SMS ou push.

Intention	Séparer deux axes de variation indépendants pour éviter l'explosion combinatoire des sous-types.
Exemple concret	Une Notification varie par type (alerte/rapport) et par canal (mail/SMS/push).
Transposition	C : Struct d'abstraction contenant un pointeur vers une interface d'implémentation. C++ : Composition entre abstraction et implementor.

Mini-exemple C

```
typedef struct { void (*send)(void *ctx, const char *msg); void *ctx; } Channel;
typedef struct { Channel channel; } Notification;
void alert_send(Notification *n, const char *m) { n->channel.send(n->channel.ctx, m); }
```

Mini-exemple C++

```
struct Notification {
    Channel& channel;
    virtual void send(Message m) = 0;
};
```

Composite

Idée en une phrase

Traiter de manière uniforme un objet simple et un groupe d'objets organisés en arbre.

Problème résolu

Le client doit parcourir une structure hiérarchique et appliquer la même opération aux feuilles et aux groupes, sans multiplier les tests de type.

Comment le pattern fonctionne

Feuilles et composites partagent un contrat commun. Le composite contient des enfants et implémente l'opération en la propageant ou en agrégeant leurs résultats.

Quand l'utiliser

Arbres de fichiers, widgets UI, expressions, scènes, règles booléennes, menus, structures organisationnelles.

Points de vigilance

Le contrat commun doit rester sémantiquement valable pour toutes les feuilles. N'ajoutez pas des méthodes absurdes uniquement pour uniformiser l'API.

Transposition en C

En C, un champ de fonction commun au début de chaque type et des casts contrôlés permettent une pseudo-hiérarchie.

Transposition en C++

En C++, une interface Node avec virtual cost() et des std::unique_ptr<Node> rend le modèle naturel.

Lecture de l'exemple

Exemple : Tache retourne son coût ; Groupe additionne le coût de tous ses enfants. Le client appelle cost() sans connaître la forme de l'arbre.

Intention	Traiter uniformément un objet simple et un groupe d'objets dans une structure arborescente.
------------------	---

Exemple concret

Calculer le coût d'un projet composé de tâches simples et de groupes de tâches.

Transposition

C : Interface commune via vtable; le composite possède une liste d'enfants.
C++ : Base Component + Leaf + Composite.

Mini-exemple C

```
typedef struct Node Node;
struct Node { int (*cost)(Node *); };
typedef struct { Node base; Node **children; size_t n; } Group;
int group_cost(Node *n) {
    Group *g=(Group*)n; int total=0;
    for(size_t i=0;i<g->n;i++) total += g->children[i]->cost(g->children[i]);
    return total;
}
```

Mini-exemple C++

```
struct Node { virtual int cost() const = 0; };
struct Group : Node {
    std::vector<std::unique_ptr<Node>> children;
    int cost() const override { int s=0; for(auto& c:children) s+=c->cost(); return s; }
};
```

Decorator

Idée en une phrase

Ajouter dynamiquement une responsabilité autour d'un objet tout en conservant la même interface.

Problème résolu

Les options sont combinables et l'héritage produit une explosion de classes : CachedLoggedEncryptedStorage, LoggedStorage, etc.

Comment le pattern fonctionne

Le décorateur implémente le même contrat que l'objet décoré, contient un delegate et exécute un comportement avant/après la délégation. Plusieurs décorateurs peuvent s'empiler.

Quand l'utiliser

Cache, logs, métriques, chiffrement, compression, retry, validation, autorisation, instrumentation.

Points de vigilance

Une chaîne trop profonde devient difficile à diagnostiquer. L'ordre des décorateurs peut aussi changer la sémantique : retry(cache(x)) n'est pas toujours équivalent à cache(retry(x)).

Transposition en C

En C, on enveloppe souvent un couple fonction/ctx avec un nouveau ctx qui contient le delegate.

Transposition en C++

En C++, un décorateur détient std::unique_ptr<Interface> ou une référence vers l'objet décoré.

Lecture de l'exemple

Exemple : CachedStorage intercepte load(), consulte le cache, puis délègue à inner_ seulement en cas de miss.

Intention

Ajouter dynamiquement une responsabilité en enveloppant un composant qui garde le même contrat.

Exemple concret

Un Storage de base peut être décoré par cache, métriques et chiffrement selon les besoins.

Transposition

C : Wrapper stockant l'interface interne et réexposant la même fonction.
C++ : Objet décorateur implémentant la même interface et contenant un delegate.

Mini-exemple C

```
typedef struct Storage Storage;
struct Storage { void *ctx; int (*load)(void*, int key); };
int metrics_load(void *ctx, int key) {
    MetricsStorage *m=ctx; m->calls++;
    return m->inner.load(m->inner.ctx, key);
}
```

Mini-exemple C++

```
class CachedStorage : public Storage {
    std::unique_ptr<Storage> inner_;
public:
    Value load(Key k) override { return cache_.getOr(k,[&]{return inner_->load(k);}); }
};
```

Facade

Idée en une phrase

Fournir une entrée simple et orientée cas d'usage vers un sous-système complexe.

Problème résolu

Le client doit connaître trop de composants internes et leur ordre d'appel. Chaque évolution du sous-système fuit vers ses consommateurs.

Comment le pattern fonctionne

La facade expose quelques opérations de haut niveau et coordonne les composants internes. Elle simplifie l'usage courant sans nécessairement interdire l'accès avancé au sous-système.

Quand l'utiliser

SDK, sous-système multimédia, pipeline de compilation, checkout, API de bibliothèque ou couche de service devant plusieurs composants techniques.

Points de vigilance

Une facade ne doit pas absorber toutes les règles métier et devenir un God Object. Elle orchestre et délègue.

Transposition en C

En C, une fonction de haut niveau peut parfaitement jouer le rôle de facade sans structure dédiée.

Transposition en C++

En C++, une petite classe facade peut regrouper les dépendances internes et exposer une API stable.

Lecture de l'exemple

Exemple : checkout() réserve le stock, débite le paiement, persiste la commande et publie un événement ; l'UI ne connaît pas ces quatre services.

Intention

Fournir une interface simple à un sous-système complexe.

Exemple concret

CheckoutFacade orchestre stock, paiement, commande et événement pour exposer une seule opération checkout().

Transposition

C : Fonction/module de façade qui appelle plusieurs sous-modules.
C++ : Classe façade composée de services internes.

Mini-exemple C

```
int checkout(Cart *c) {
    if (!stock_reserve(c)) return 0;
    if (!payment_charge(c->total)) return 0;
    order_save(c);
    event_publish("checkout.done");
    return 1;
}
```

Mini-exemple C++

```
class CheckoutFacade {
public:
    Receipt checkout(const Cart& c) {
        stock_.reserve(c); auto p=pay_.charge(c.total()); orders_.save(c,p); return {p.id};
    }
};
```

Point de vigilance - La façade ne doit pas devenir le nouveau God Object contenant toutes les règles métier.

Flyweight

Idée en une phrase

Partager un état intrinsèque commun entre un très grand nombre d'objets afin de réduire la mémoire.

Problème résolu

Des milliers ou millions d'objets répètent exactement les mêmes données lourdes : police, style, mesh, métadonnées, configuration immuable.

Comment le pattern fonctionne

L'état commun est interné dans un objet partagé (flyweight). Chaque instance légère ne garde que son état extrinsèque propre et une référence vers le flyweight.

Quand l'utiliser

Glyphes de texte, particules, tuiles, arbres de syntaxe, modèles 3D partagés, chaînes internées.

Points de vigilance

C'est d'abord un pattern d'optimisation. Mesurez la volumétrie avant de complexifier le modèle ; attention à la durée de vie du cache de flyweights.

Transposition en C

En C, un pool/intern table retourne des pointeurs const vers des structures partagées.

Transposition en C++

En C++, `std::shared_ptr<const Style>` ou des handles vers un cache central permettent le partage contrôlé.

Lecture de l'exemple

Exemple : chaque Glyph stocke seulement le caractère et un pointeur vers un Style partagé par toutes les lettres ayant la même police/taille/couleur.

Intention

Partager l'état intrinsèque commun entre un grand nombre d'objets afin de réduire la mémoire.

Exemple concret

Un éditeur de texte partage un objet GlyphStyle entre des milliers de caractères ayant la même police/couleur.

Transposition

C : Table d'interning renvoyant un pointeur vers une structure immuable partagée.

C++ : `std::shared_ptr<const Style>` obtenu depuis un pool/intern table.

Mini-exemple C

```
typedef struct { int font; int size; int color; } GlyphStyle;
const GlyphStyle *style_intern(StylePool *p, GlyphStyle wanted);
typedef struct { char ch; const GlyphStyle *style; } Glyph;
```

Mini-exemple C++

```
struct Glyph { char32_t ch; std::shared_ptr<const Style> style; };
auto style = styles.intern({"Inter",12,Black});
```

Point de vigilance - Flyweight partage des données; Object Pool réutilise des instances au cours du temps. Les intentions sont différentes.

Proxy

Idée en une phrase

Placer un représentant devant un objet afin de contrôler ou différer l'accès à cet objet.

Problème résolu

L'objet réel est coûteux à créer, distant, protégé, soumis à des droits ou nécessite un cache. Le client doit conserver le même contrat.

Comment le pattern fonctionne

Le proxy implémente la même interface que le sujet réel et intercepte les appels. Il peut créer l'objet à la demande, vérifier les droits, faire un appel réseau ou servir une valeur en cache.

Quand l'utiliser

Lazy loading, remote proxy, protection proxy, virtual proxy, cache proxy, smart references.

Points de vigilance

Un proxy distant ne rend pas un appel réseau identique à un appel local : latence, timeouts et erreurs doivent rester visibles dans le contrat si elles sont significatives.

Transposition en C

En C, une structure proxy contient le handle de l'objet réel et éventuellement l'information nécessaire pour le créer.

Transposition en C++

En C++, le proxy dérive souvent de la même interface et initialise le sujet réel à la première utilisation.

Lecture de l'exemple

Exemple : DocumentProxy ne charge le gros document depuis le disque qu'au premier appel à `content()`.

Intention

Contrôler l'accès à un objet en conservant son contrat.

Exemple concret

Un DocumentProxy retarde le chargement d'un gros fichier jusqu'au premier accès à `content()`.

Transposition

C : Wrapper avec lazy init, contrôle de droit ou indirection distante.

C++ : Classe Proxy ayant la même interface que l'objet réel.

Mini-exemple C

```
typedef struct { Document *real; const char *path; } DocumentProxy;
const char *proxy_content(DocumentProxy *p) {
    if (!p->real) p->real = document_load(p->path);
    return document_content(p->real);
}
```

Mini-exemple C++

```
class DocumentProxy : public Document {
    std::unique_ptr<RealDocument> real_;
public:
    Bytes content() override { if(!real_) real_=load(); return real_->content(); }
};
```

4. Patterns comportementaux

Chain of Responsibility

Idée en une phrase

Faire traverser une requête à une suite de handlers sans que l'émetteur connaisse celui qui la traitera finalement.

Problème résolu

Le code contient une succession rigide de if/else ou le client doit connaître tous les validateurs, filtres ou niveaux d'escalade.

Comment le pattern fonctionne

Chaque handler peut traiter, refuser ou transmettre au suivant. La chaîne est composée séparément de la logique cliente.

Quand l'utiliser

Middleware HTTP, validations, filtres, logging, contrôle d'accès, escalade de support, traitement de protocoles.

Points de vigilance

Une chaîne trop opaque rend l'ordre difficile à comprendre. Le contrat doit préciser quand la chaîne s'arrête et comment les erreurs sont propagées.

Transposition en C

En C, chaque Handler peut contenir un pointeur vers next et une fonction handle.

Transposition en C++

En C++, `std::unique_ptr<Handler> next` permet une ownership claire ; des lambdas peuvent aussi former une chaîne légère.

Lecture de l'exemple

Exemple : AuthHandler vérifie l'autorisation ; si elle est valide, il délègue au handler suivant, sinon il retourne 403.

Intention

Faire circuler une requête dans une chaîne de handlers jusqu'à traitement, refus ou transmission.

Exemple concret

Pipeline HTTP : authentication -> rate limit -> validation -> handler métier.

Transposition

C : Chaque handler possède fonction pointer + next.

C++ : Objets Handler chaînés, ou composition de fonctions/middlewares.

Mini-exemple C

```
typedef struct Handler Handler;
struct Handler { int (*handle)(Handler*, Request*); Handler *next; };
int auth_handle(Handler *h, Request *r) {
    if (!authorized(r)) return 403;
    return h->next->handle(h->next, r);
}
```

Mini-exemple C++

```
struct Handler {
    std::unique_ptr<Handler> next;
    virtual Result handle(Request r) { return next ? next->handle(r) : Result{}; }
};
```

Command

Idée en une phrase

Représenter une action comme un objet ou une structure de données manipulable.

Problème résolu

Une action doit être mise en file, planifiée, journalisée, rejouée, annulée, sérialisée ou traitée indépendamment de l'émetteur.

Comment le pattern fonctionne

La commande encapsule l'opération et ses paramètres. Un invoker stocke ou déclenche la commande ; un receiver effectue réellement le travail.

Quand l'utiliser

Undo/redo, UI, jobs, queues, CQRS, scripts, traitement asynchrone, macro-commandes.

Points de vigilance

Ne créez pas une classe Command pour chaque appel banal. Le pattern prend de la valeur lorsque la requête possède un cycle de vie propre.

Transposition en C

En C, un couple ctx + execute/undo est suffisant ; la commande peut être une struct triviale sérialisable.

Transposition en C++

En C++, interface Command, std::function<void()> ou type erased callable sont possibles selon les besoins.

Lecture de l'exemple

Exemple : SaveFileCommand est placé dans une queue de travail puis exécuté plus tard ; l'émetteur ne connaît pas le thread ni le moment d'exécution.

Intention

Encapsuler une action et ses paramètres dans un objet/structure afin de pouvoir la stocker, rejouer, annuler ou mettre en file.

Exemple concret

Une UI crée des commandes SaveFile et Deleteltem que l'historique peut rejouer ou annuler.

Transposition

C : Struct {execute, undo, ctx}.

C++ : Objet Command ou lambda capturant les paramètres.

Mini-exemple C

```
typedef struct { void *ctx; void (*execute)(void*); void (*undo)(void*); } Command;
void run(Command c) { c.execute(c.ctx); }
```

Mini-exemple C++

```
struct Command { virtual void execute()=0; virtual void undo()=0; };
queue.push(std::make_unique<SaveFileCommand>(file));
```

Interpreter

Idée en une phrase

Représenter les règles d'une petite grammaire par des objets/structures capables de s'évaluer.

Problème résolu

L'application doit comprendre un mini-langage simple : filtres, règles, expressions booléennes ou DSL très limité.

Comment le pattern fonctionne

Chaque règle grammaticale devient un type d'expression. Les expressions terminales portent les valeurs et les non-terminales combinent d'autres expressions pour former un arbre.

Quand l'utiliser

Règles simples, filtres de recherche, expressions mathématiques limitées, petits DSL embarqués.

Points de vigilance

La complexité augmente vite avec une grammaire riche. Pour un vrai langage, utilisez un parser dédié, un générateur de parser ou une AST séparée.

Transposition en C

En C, un enum de kind et une union/structure d'AST permettent d'implémenter eval avec un switch central ou des pointeurs de fonction.

Transposition en C++

En C++, une hiérarchie Expr ou std::variant + std::visit sont deux options naturelles.

Lecture de l'exemple

Exemple : And(GreaterThan(x, 10), Const(true)) forme un arbre d'expression évalué contre un Context.

Intention

Représenter une petite grammaire sous forme d'objets/nœuds et évaluer une expression.

Exemple concretUn moteur de règles évalue `country == "FR" AND total > 100`.**Transposition**

C : AST taggé + fonction eval récursive.

C++ : Hiérarchie Expr, `std::variant/std::visit` ou parser combinators.**Mini-exemple C**

```
typedef enum { EXPR_AND, EXPR_GT, EXPR_CONST } ExprKind;
typedef struct Expr Expr;
struct Expr { ExprKind kind; Expr *a,*b; int value; };
int eval(const Expr *e) { return e->kind==EXPR_AND ? eval(e->a)&&eval(e->b) : e->value; }
```

Mini-exemple C++

```
struct Expr { virtual bool eval(const Context&) const = 0; };
struct And : Expr { std::unique_ptr<Expr> a,b; bool eval(const Context& c) const override { return a->eval(c)&&b->eval(c); } };
```

Point de vigilance - Pour une grammaire riche, utiliser un vrai parseur est souvent préférable.

Iterator

Idée en une phrase

Parcourir une collection sans exposer sa représentation interne.

Problème résolu

Le client doit accéder à une structure complexe ou plusieurs modes de parcours sont nécessaires. Exposer tableaux, buckets ou pointeurs internes crée un couplage fort.

Comment le pattern fonctionne

L'itérateur conserve l'état du parcours et expose une opération next/current ou les opérateurs d'itération du langage.

Quand l'utiliser

Collections, arbres, graphes, flux, séquences paresseuses, structures custom.

Points de vigilance

Précisez la validité de l'itérateur lors des modifications de la collection. L'invalidation est souvent le vrai sujet en C/C++.

Transposition en C

En C, un struct Iterator contient l'index, les pointeurs ou l'état nécessaire ; next renvoie false en fin de séquence.

Transposition en C++

En C++, suivre les conventions STL rend le type compatible avec for-range et les algorithmes standards.

Lecture de l'exemple

Exemple : MapIter masque le tableau de buckets d'une hash map et retourne successivement les couples clé/valeur.

Intention

Parcourir une collection sans exposer sa représentation interne.

Exemple concret

Une table de hash interne fournit un itérateur key/value sans révéler ses buckets.

Transposition

C : Struct iterator contenant index/état + next().

C++ : Itérateurs STL, ranges, generator ou itérateur custom.

Mini-exemple C

```
typedef struct { Map *map; size_t bucket; Entry *cur; } MapIter;
bool map_iter_next(MapIter *it, Key *k, Value *v);
```

Mini-exemple C++

```
for (auto it = map.begin(); it != map.end(); ++it) {
    use(it->first, it->second);
}
```

Mediator

Idée en une phrase

Centraliser la coordination entre plusieurs objets afin d'éviter un graphe dense de dépendances mutuelles.

Problème résolu

De nombreux collègues se connaissent directement et chaque changement d'interaction modifie plusieurs classes. L'UI est un cas classique.

Comment le pattern fonctionne

Les collègues communiquent avec le médiateur. Celui-ci connaît le protocole global et déclenche les réactions entre participants.

Quand l'utiliser

Boîtes de dialogue, workflows, orchestration locale, composants UI, coordination de modules fortement interactifs.

Points de vigilance

Le mediator peut devenir un God Object. Gardez les règles locales dans les objets experts et limitez le médiateur à la coordination.

Transposition en C

En C, une structure de contexte central et des fonctions callback suffisent.

Transposition en C++

En C++, les widgets peuvent posséder une référence vers un DialogMediator qui traite leurs événements.

Lecture de l'exemple

Exemple : quand la case "mode avancé" change, le mediator active le bouton et affiche le panneau sans que les widgets se réfèrent directement.

Intention	Centraliser la coordination entre plusieurs collègues pour éviter un graphe dense de références mutuelles.
Exemple concret	Dans une boîte de dialogue, le changement de la checkbox Advanced active/désactive plusieurs widgets via DialogMediator.
Transposition	C : Chaque composant connaît un mediator et lui signale les événements. C++ : Objet Mediator recevant les événements et coordonnant les collègues.

Mini-exemple C

```
typedef struct DialogMediator DialogMediator;
void checkbox_changed(DialogMediator *m, int checked) {
    button_set_enabled(m->advanced_button, checked);
    panel_set_visible(m->advanced_panel, checked);
}
```

Mini-exemple C++

```
class DialogMediator {
public:
    void changed(Widget& w) { /* coordonne les autres widgets */ }
};
```

Memento

Idée en une phrase

Capturer l'état d'un objet afin de pouvoir le restaurer plus tard sans exposer directement ses détails internes.

Problème résolu

Il faut implémenter undo, checkpoints ou rollback, mais donner au caretaker un accès complet à l'état interne casserait l'encapsulation.

Comment le pattern fonctionne

L'originator fabrique un snapshot opaque. Un caretaker le stocke sans en interpréter le contenu, puis le redonne à l'originator pour restauration.

Quand l'utiliser

Éditeurs, jeux, transactions locales, assistants multi-étapes, snapshots de configuration.

Points de vigilance

Les snapshots peuvent être très coûteux. Pour un gros état, envisagez des deltas, copy-on-write ou un journal d'événements.

Transposition en C

En C, le memento est une structure copiée/allouée contenant uniquement l'état nécessaire à la restauration.

Transposition en C++

En C++, il peut être un type value privé/imbriqué retourné par snapshot() et consommé par restore().

Lecture de l'exemple

Exemple : l'éditeur mémorise texte + position du curseur avant replaceAll(), puis restore() annule l'opération.

Intention

Capturer un état pour pouvoir le restaurer plus tard sans exposer les détails internes.

Exemple concret

Un éditeur de texte sauvegarde un snapshot avant une opération afin d'implémenter Undo.

Transposition

C : Fonction snapshot/restore renvoyant une structure opaque copiée.
C++ : Objet Memento opaque, copie d'état, ou value snapshot.

Mini-exemple C

```
typedef struct { char *text; size_t cursor; } EditorMemento;
EditorMemento editor_snapshot(const Editor *e);
void editor_restore(Editor *e, const EditorMemento *m);
```

Mini-exemple C++

```
auto m = editor.snapshot();
editor.replaceAll("x","y");
editor.restore(m);
```

Observer

Idée en une phrase

Notifier plusieurs abonnés lorsqu'un fait ou un changement d'état se produit.

Problème résolu

Un producteur ne doit pas connaître à l'avance toutes les réactions possibles. Ajouter Analytics, UI ou logging ne devrait pas modifier le producteur.

Comment le pattern fonctionne

Les observateurs s'abonnent. Le sujet publie un événement et parcourt les abonnés. Dans une version asynchrone, un bus peut découpler aussi le temps d'exécution.

Quand l'utiliser

UI, événements métier, notifications locales, caches, métriques, plugins, réactions secondaires.

Points de vigilance

Gérez désabonnement, durée de vie, ordre, exceptions et réentrance. Dans un système distribué, Observer ne suffit pas à résoudre livraison, duplication et persistance.

Transposition en C

En C, un Subscriber contient fn + ctx ; le bus stocke un tableau/liste de subscribers.

Transposition en C++

En C++, callbacks, std::function, signaux/slots ou event bus typé sont fréquents.

Lecture de l'exemple

Exemple : OrderConfirmed est publié ; Analytics et EmailListener réagissent sans être connus par la commande.

Intention

Notifier plusieurs abonnés lorsqu'un événement ou un changement survient.

Exemple concret

Après OrderConfirmed, email, analytics et fidélité réagissent indépendamment.

Transposition

C : Liste de callbacks {fn,ctx}.
C++ : Callbacks, signals, event bus local, std::function.

Mini-exemple C

```
typedef void (*EventFn)(void *ctx, const Event *e);
typedef struct { EventFn fn; void *ctx; } Subscriber;
void publish(Bus *b, Event *e) {
    for(size_t i=0; i<b->n; i++) b->subs[i].fn(b->subs[i].ctx, e);
}
```

Mini-exemple C++

```
bus.subscribe<OrderConfirmed>([&](auto e){ analytics.record(e); });
```

Point de vigilance - Prévoir unsubscribe/lifetime, ordre, erreurs et comportement sync/async.

State

Idée en une phrase

Déplacer les comportements dépendant de l'état dans des objets d'état plutôt que disperser des if/switch partout.

Problème résolu

Une entité possède plusieurs états et transitions ; chaque méthode commence à tester `state == X`, ce qui rend les règles de cycle de vie difficiles à maintenir.

Comment le pattern fonctionne

Le contexte délègue au State courant. Une transition remplace l'objet State et modifie ainsi le comportement futur.

Quand l'utiliser

Connexions réseau, workflow de commande, lecteur multimédia, machine à états de protocole, UI complexe.

Points de vigilance

Pour deux états simples, un enum est souvent plus clair. Le pattern devient utile lorsque les comportements et transitions sont nombreux et dispersés.

Transposition en C

En C, l'état peut être un pointeur vers une table de fonctions StateVTable.

Transposition en C++

En C++, interface ConnState + objets Connected/Disconnected, ou `std::variant` pour une approche plus statique.

Lecture de l'exemple

Exemple : `send()` n'a pas le même comportement en Disconnected et Connected ; changer l'état remplace la stratégie interne du contexte.

Intention

Faire varier le comportement selon l'état interne d'un objet et modéliser explicitement les transitions.

Exemple concret

Une connexion est Disconnected, Connecting ou Connected; `send()` n'a pas le même comportement selon l'état.

Transposition

C : Etat représenté par table de fonctions ou enum + dispatch.

C++ : Objets State, `std::variant`, ou enum/switch si la machine reste petite.

Mini-exemple C

```
typedef struct State State;
struct State { int (*send)(Connection*, const char*); };
int disconnected_send(Connection *c, const char *m) { return ERR_NOT_CONNECTED; }
```

Mini-exemple C++

```
struct Connected : ConnState {
    Result send(Connection& c, Message m) override { return c.socket().write(m); }
};
```

Point de vigilance - Un enum + switch est souvent plus clair pour quelques états stables.

Strategy

Idée en une phrase

Encapsuler une famille d'algorithmes interchangeables derrière un contrat commun.

Problème résolu

Un if/switch choisit entre plusieurs politiques qui évoluent séparément : tarification, comparaison, routage, compression, scoring.

Comment le pattern fonctionne

Le contexte reçoit une stratégie et lui délègue la partie variable. La stratégie peut être choisie à la composition, par configuration ou dynamiquement.

Quand l'utiliser

Quand plusieurs politiques réelles existent, doivent être testées séparément ou ajoutées sans modifier le contexte.

Points de vigilance

Un switch local de trois lignes n'a pas toujours besoin de Strategy. Le pattern est justifié si la variation possède sa propre logique et son propre rythme d'évolution.

Transposition en C

En C, un pointeur de fonction + ctx est souvent la forme la plus idiomatique et très légère.

Transposition en C++

En C++, interface virtuelle, template policy, lambda ou callable générique permettent différents compromis entre runtime et compile-time.

Lecture de l'exemple

Exemple : `sort_items` reçoit une CompareFn ; le tri reste stable tandis que la politique de comparaison varie.

Intention

Encapsuler un algorithme interchangeable derrière un même contrat.

Exemple concret

Un tri utilise au choix ordre lexical, longueur ou score métier.

Transposition

C : Pointeur de fonction + contexte.

C++ : Objet Strategy, lambda, functor ou template policy.

Mini-exemple C

```
typedef int (*CompareFn)(const Item*, const Item*, void *ctx);
void sort_items(Item *a, size_t n, CompareFn cmp, void *ctx);
```

Mini-exemple C++

```
std::sort(items.begin(), items.end(), [](auto& a, auto& b){ return a.score < b.score; });
```

Template Method

Idée en une phrase

Fixer la séquence générale d'un algorithme tout en laissant certaines étapes spécialisables.

Problème résolu

Plusieurs variantes partagent le même workflow mais diffèrent sur quelques étapes. Copier le workflow crée de la duplication et un risque de divergence.

Comment le pattern fonctionne

Le squelette appelle des hooks. En C++ classique, le squelette est final/non virtuel et les étapes sont virtuelles. En C, des callbacks jouent le même rôle.

Quand l'utiliser

Import/export, parsing, pipelines synchrones, génération de rapport, workflows avec ordre stable.

Points de vigilance

Le pattern crée un couplage fort au squelette. Si les étapes doivent être recombinaées librement, Strategy/Composition est souvent plus flexible.

Transposition en C

En C, une structure ImportHooks contient les fonctions variables tandis que import_run impose l'ordre.

Transposition en C++

En C++, une classe base possède run() et appelle des méthodes protected virtuelles.

Lecture de l'exemple

Exemple : importer() impose lire -> valider -> convertir -> sauvegarder ; seuls lire et valider changent selon le format.

Intention

Fixer le squelette d'un algorithme tout en laissant certaines étapes variables.

Exemple concret

Tous les imports suivent lire -> valider -> convertir -> sauvegarder, mais lire et valider dépendent du format.

Transposition

C : Fonction template recevant des callbacks pour les étapes variables.

C++ : Classe mère avec méthode final/non virtuelle orchestrant des hooks, ou composition.

Mini-exemple C

```
typedef struct { Data (*read)(Source); bool (*validate)(Data); } ImportHooks;
void import_run(Source s, ImportHooks h) {
    Data d=h.read(s); if(h.validate(d)) save(convert(d));
}
```

Mini-exemple C++

```
struct Importer {
    void run(Source s) { auto d=read(s); if(validate(d)) save(convert(d)); }
    virtual Data read(Source)=0; virtual bool validate(const Data&)=0;
};
```

Visitor

Idée en une phrase

Ajouter de nouvelles opérations à une structure de types relativement stable sans mettre toutes ces opérations dans les classes visitées.

Problème résolu

Une hiérarchie AST ou modèle possède de nombreux types. On veut ajouter export, validation, pretty-print, métriques, etc., et chaque opération dépend du type concret.

Comment le pattern fonctionne

Chaque élément accepte un Visitor. Le double dispatch appelle visit(TypeConcret), ce qui sélectionne le comportement selon le visiteur ET le type visité.

Quand l'utiliser

AST de compilateur, modèles de documents, structures géométriques, arbres dont les types changent peu mais les opérations s'ajoutent souvent.

Points de vigilance

Ajouter un nouveau type d'élément oblige à modifier tous les visiteurs. Visitor échange donc facilité d'ajout d'opérations contre coût d'évolution de la hiérarchie.

Transposition en C

En C, une table de fonctions visitor + un kind de noeud simule le double dispatch.

Transposition en C++

En C++, interface Visitor et accept() est classique ; std::variant + std::visit offre une alternative statique.

Lecture de l'exemple

Exemple : Number et Add acceptent un PrettyPrinterVisitor puis un EvaluatorVisitor sans stocker ces responsabilités dans les noeuds.

Intention

Ajouter de nouvelles opérations à une structure de types relativement stable sans modifier chaque algorithme client.

Exemple concret

Un AST possède Number, Add et Call; on ajoute pretty-print, analyse et génération de code comme visiteurs.

Transposition

C : Tag de type + table de fonctions visit_x.

C++ : Double dispatch classique ou std::variant + std::visit.

Mini-exemple C

```
typedef struct Visitor Visitor;
struct Visitor { void (*number)(Visitor*, Number*); void (*add)(Visitor*, Add*); };
void node_accept(Node *n, Visitor *v) {
    if(n->kind==NUMBER) v->number(v, (Number*)n); else v->add(v, (Add*)n);
}
```

Mini-exemple C++

```
struct Visitor { virtual void visit(Number&)=0; virtual void visit(Add&)=0; };
struct Node { virtual void accept(Visitor&)=0; };
```

Point de vigilance - Visitor facilite l'ajout d'opérations mais rend plus coûteux l'ajout de nouveaux types de nœuds.

5. Object Pool - exemple complet

Idée en une phrase

Réutiliser un ensemble borné d'objets déjà réservés afin de rendre acquisition/libération rapides et prévisibles.

Problème résolu

Créer/détruire constamment certains objets est coûteux : connexions, buffers réseau, particules, commandes, objets graphiques. L'allocation générale peut aussi fragmenter le heap ou introduire une latence non déterministe.

Comment le pattern fonctionne

Le pool possède N slots. acquire() prend un slot libre ; release() le remet disponible. Selon le cas, l'objet reste construit entre deux usages ou est reconstruit in-place. Une free-list évite de scanner tous les slots.

Quand l'utiliser

Systèmes temps réel, moteurs de jeu, réseau, embarqué, serveurs fortement chargés et toute ressource où une capacité maximale raisonnable peut être définie.

Points de vigilance

Définissez absolument la politique d'épuisement : retourner null/optional, bloquer, attendre avec timeout ou agrandir. Évitez les use-after-release ; avec des handles, une génération aide à détecter les références périmées.

Transposition en C

En C, un tableau statique + free-list d'indices est simple, rapide et sans fragmentation. reset de l'objet doit être explicite avant réutilisation.

Transposition en C++

En C++, une Lease RAII peut rendre automatiquement l'objet au pool. Pour des types non trivialement réinitialisables, utilisez placement new / std::construct_at et std::destroy_at avec précaution.

Lecture de l'exemple

Exemple : un pool de 64 PacketBuffer de 1 KiB préalloue la mémoire. Chaque traitement réseau emprunte un buffer puis le restitue, sans malloc dans le chemin chaud.

Object Pool réutilise un ensemble borné d'objets. Le cas classique en C/C++ est un buffer réseau, une connexion ou une entité de jeu coûteuse à créer. L'exemple ci-dessous montre un pool fixe de PacketBuffer : aucune allocation après l'initialisation, acquisition O(1), restitution O(1).

C - Object Pool fixe avec free-list

```
#define CAP 64
#define NONE 0xFFFFu

typedef struct {
    unsigned char data[1024];
    size_t used;
} PacketBuffer;

typedef struct {
    PacketBuffer objects[CAP];
    unsigned short next[CAP];
    unsigned short free_head;
} PacketPool;

void pool_init(PacketPool *p) {
    for (unsigned short i=0; i<CAP-1; ++i) p->next[i]=i+1;
    p->next[CAP-1]=NONE;
    p->free_head=0;
}

PacketBuffer *pool_acquire(PacketPool *p) {
    if (p->free_head==NONE) return NULL;
    unsigned short i=p->free_head;
    p->free_head=p->next[i];
    p->objects[i].used=0;
    return &p->objects[i];
}

void pool_release(PacketPool *p, PacketBuffer *b) {
    unsigned short i=(unsigned short)(b-p->objects);
    p->next[i]=p->free_head;
    p->free_head=i;
}
```

C++ - Lease RAII : le retour au pool est automatique

```
template<class T, std::size_t N>
class ObjectPool {
    std::array<T,N> objects_{};
    std::array<bool,N> used_{};
public:
    class Lease {
        ObjectPool *pool_{}; T *ptr_{};
    public:
        Lease(ObjectPool& p, T& x) : pool_(&p), ptr_(&x) {}
        Lease(Lease&& o) noexcept : pool_(std::exchange(o.pool_,nullptr)), ptr_(std::exchange(o.ptr_,nullptr)) {}
        ~Lease() { if (pool_) pool_->release(*ptr_); }
        T* operator->() { return ptr_; }
    };

    std::optional<Lease> acquire() {
        for (std::size_t i=0;i<N;++i) if(!used_[i]) {
            used_[i]=true; objects_[i].reset(); return Lease{*this,objects_[i]};
        }
        return std::nullopt;
    }

    void release(T& x) { used_[&x-objects_.data()] = false; }
};
```

Exemple concret : un serveur traite 20 000 paquets/s. Chaque requête a besoin temporairement d'un buffer de 1 KiB. Préallouer 256 buffers évite des milliers de malloc/free par seconde et permet de borner la mémoire. Il faut néanmoins mesurer le high-water mark, définir la politique d'épuisement et garantir un reset complet avant réutilisation.

6. Mémoire et ressources

Memory Pool / Fixed Block Allocator

Idée en une phrase

Réutiliser des blocs mémoire de taille fixe afin d'éviter les coûts et la fragmentation de malloc/free répétés.

Problème résolu

Une application alloue énormément de petits objets de même taille dans une boucle chaude ou un contexte temps réel.

Comment le pattern fonctionne

Un ensemble de blocs préalloués est chaîné dans une free-list. alloc retire un bloc libre ; free le remet dans la liste. Le coût est généralement $O(1)$.

Quand l'utiliser

Paquets réseau, nodes de liste, messages, objets de jeu homogènes, systèmes embarqués.

Points de vigilance

Le pool est spécialisé sur une taille et une capacité. Il faut définir la politique d'épuisement et éviter les doubles free.

Transposition en C

En C, une union Block contenant next ou bytes permet d'éviter tout metadata externe.

Transposition en C++

En C++, `std::pmr::unsynchronized_pool_resource` fournit un pool générique ; un allocator custom peut cibler un type précis.

Lecture de l'exemple

Exemple : 512 blocs de 64 octets sont préparés ; chaque allocation retire simplement la tête de `free_list`.

Intention

Préallouer du stockage découpé en blocs de taille fixe afin de rendre allocation/libération rapides et prévisibles.

Exemple concret

Un firmware réserve 512 blocs de 64 octets pour de petits messages sans appeler malloc après boot.

Transposition

C : Free-list d'indices ou de pointeurs.
C++ : Allocator custom, `std::pmr` ou `pool resource`.

Mini-exemple C

```
typedef union Block { union Block *next; unsigned char bytes[64]; } Block;
static Block blocks[512]; static Block *free_list;
void *block_alloc(void){ Block *b=free_list; if(b) free_list=b->next; return b; }
void block_free(void *p){ Block *b=p; b->next=free_list; free_list=b; }
```

Mini-exemple C++

```
std::pmr::unsynchronized_pool_resource pool;
std::pmr::vector<int> v{&pool};
```

Arena / Region Allocator

Idée en une phrase

Allouer très rapidement de nombreux objets ayant approximativement la même durée de vie, puis tout libérer d'un coup.

Problème résolu

Des milliers de petites allocations sont créées pendant une requête, une frame ou un parsing et seront toutes détruites ensemble.

Comment le pattern fonctionne

Une grande zone mémoire possède un offset courant. Chaque allocation avance l'offset avec l'alignement requis. `reset()` remet l'offset à zéro sans parcourir les objets.

Quand l'utiliser

Compilateurs, AST, moteurs de jeu par frame, traitements batch, parsing, requêtes serveur.

Points de vigilance

Les destructeurs individuels ne sont pas automatiquement appelés dans une arena brute. Ne laissez pas de pointeurs survivre au reset.

Transposition en C

En C, quelques lignes avec base/capacity/offset suffisent à construire une bump allocator très rapide.

Transposition en C++

En C++, `std::pmr::monotonic_buffer_resource` fournit exactement ce modèle pour les conteneurs PMR.

Lecture de l'exemple

Exemple : tous les Node d'un AST sont créés dans une arena ; à la fin de la compilation du fichier, on reset la région en une opération.

Intention

Effectuer de nombreuses allocations rapides puis libérer toute une région d'un coup.

Exemple concret

Un parseur construit un AST pour une requête; tout l'AST peut être libéré en fin de requête.

Transposition

C : Bump pointer dans un buffer; reset de l'offset.

C++ : `std::pmr::monotonic_buffer_resource` ou arena custom.

Mini-exemple C

```
void *arena_alloc(Arena *a, size_t n, size_t align) {
    size_t p=(a->offset + align-1) & ~(align-1);
    if (p+n>a->capacity) return NULL;
    void *r=a->base+p; a->offset=p+n; return r;
}
void arena_reset(Arena *a){ a->offset=0; }
```

Mini-exemple C++

```
std::byte buf[64*1024];
std::pmr::monotonic_buffer_resource arena{buf,sizeof buf};
std::pmr::vector<Node> nodes{&arena};
```

Slab / Object Cache

Idée en une phrase

Maintenir des slabs contenant des emplacements dédiés à un type d'objet précis, souvent pour obtenir prédictibilité et localité.

Problème résolu

Un allocator généraliste ajoute trop de metadata ou de fragmentation pour un type très fréquent et de taille connue.

Comment le pattern fonctionne

Chaque slab contient N slots alignés pour T et une structure de disponibilité. Plusieurs slabs peuvent être ajoutés si le besoin dépasse la capacité.

Quand l'utiliser

Kernels, caches de structures système, objets réseau, entités fréquentes, composants temps réel.

Points de vigilance

Gestion plus complexe que Memory Pool : croissance, libération de slabs vides, alignement et concurrence doivent être explicités.

Transposition en C

En C, un tableau d'objets + bitmap/free-list constitue une version simple.

Transposition en C++

En C++, `alignas(T)` et `std::byte` permettent de stocker les slots sans construire T avant l'acquisition.

Lecture de l'exemple

Exemple : SocketCB est un type fréquent ; chaque slab garde 128 slots et un bitmap des emplacements occupés.

Intention

Conserver des caches d'objets homogènes pour réduire fragmentation et coût d'initialisation.

Exemple concret

Un noyau maintient un cache de structures SocketControlBlock de même taille et fréquemment recyclées.

Transposition

C : Pages/slabs contenant plusieurs objets libres/occupés.

C++ : Allocator spécialisé ou cache de T avec placement new.

Mini-exemple C

```
typedef struct Slab { SocketCB objs[128]; unsigned char used[128]; } Slab;
SocketCB *socketcb_alloc(Slab *s) { /* cherche un slot libre */ return NULL; }
```

Mini-exemple C++

```
template<class T, std::size_t N>
struct Slab { alignas(T) std::byte mem[N*sizeof(T)]; std::bitset<N> used; };
```

RAII

Idée en une phrase

Lier la durée de vie d'une ressource à la durée de vie d'un objet afin que l'acquisition et surtout la libération soient automatiques.

Problème résolu

Les retours anticipés et exceptions provoquent des fuites de fichiers, mutex, sockets ou mémoire si chaque chemin doit appeler manuellement cleanup.

Comment le pattern fonctionne

Le constructeur acquiert ou reçoit la ressource ; le destructeur la libère. Le scope C++ devient alors le mécanisme de cleanup.

Quand l'utiliser

Pratiquement toutes les ressources en C++ : mémoire, fichiers, mutex, transactions, handles OS, sockets.

Points de vigilance

RAII est un idiome C++, pas un mécanisme automatique du C. Les objets RAII doivent définir clairement ownership, move et éventuellement copy.

Transposition en C

En C, on imite l'intention avec une discipline goto cleanup ou des wrappers spécifiques, mais le langage n'appelle pas automatiquement un destructeur en sortie de scope.

Transposition en C++

En C++, préférez `std::unique_ptr`, `std::lock_guard`, `std::fstream` et autres wrappers RAII avant d'écrire le vôtre.

Lecture de l'exemple

Exemple : File ouvre le fichier au constructeur ; son destructeur appelle `fclose` même si une exception interrompt le traitement.

Intention

Lier la durée de vie d'une ressource à celle d'un objet afin que la libération soit automatique à la sortie du scope.

Exemple concret

Un fichier ouvert est fermé même si une exception interrompt la fonction.

Transposition

C : Pas de destructeur automatique : utiliser une discipline cleanup/goto ou helpers scope.
C++ : Destructeur, `unique_ptr` avec `deleter`, `lock_guard`, `jthread`, etc.

Mini-exemple C

```
int work(void) {
    FILE *f=fopen("data.bin","rb"); if(!f) return -1;
    int rc=do_work(f);
    fclose(f);
    return rc;
}
```

Mini-exemple C++

```
class File {
    FILE *f_;
public:
    explicit File(const char* p):f_(std::fopen(p,"rb")){ if(!f_) throw std::runtime_error("open"); }
    ~File(){ std::fclose(f_); }
};
```

Scope Guard / defer

Idée en une phrase

Enregistrer une action de nettoyage à exécuter automatiquement à la sortie du scope.

Problème résolu

Une fonction possède une ressource ou une action de rollback qui doit s'exécuter sur tous les chemins d'erreur, sans créer un type RAII complet.

Comment le pattern fonctionne

Un guard stocke un callback ; son destructeur l'exécute sauf s'il a été annulé après succès.

Quand l'utiliser

Rollback transactionnel, restauration de flags, suppression de fichier temporaire, unlock ou actions de cleanup ponctuelles.

Points de vigilance

Le callback doit être sûr pendant le unwinding et ne pas lancer d'exception. Attention aux captures de références dont la durée de vie serait plus courte que le guard.

Transposition en C

En C, goto cleanup reste l'équivalent le plus robuste et lisible ; certains compilateurs proposent des extensions cleanup.

Transposition en C++

En C++23, std::scope_exit fournit le pattern directement ; avant cela, de petites classes ScopeGuard sont courantes.

Lecture de l'exemple

Exemple : rollback est armé avant update(); après tx.commit(), rollback.release() empêche l'annulation.

Intention

Exécuter automatiquement une action à la sortie d'un scope, utile pour rollback ou cleanup non naturellement objet.

Exemple concret

Une transaction doit rollback si commit n'a pas été exécuté.

Transposition

C : goto cleanup reste une structure idiomatique robuste.
C++ : std::scope_exit (C++23) ou guard local.

Mini-exemple C

```
int f(void){
    Resource *r=open_resource(); if(!r) return -1;
    int rc=step(r); if(rc) goto cleanup;
    rc=step2(r);
cleanup:
    close_resource(r); return rc;
}
```

Mini-exemple C++

```
auto rollback = std::scope_exit([&]{ tx.rollback(); });
update(); tx.commit(); rollback.release();
```

7. Idioms et patterns spécifiques C/C++

Opaque Pointer / Opaque Handle

Idée en une phrase

Cacher la représentation d'un objet derrière un type incomplet ou un handle public.

Problème résolu

Le header expose tous les champs internes, créant un couplage binaire/source et autorisant le client à modifier l'état directement.

Comment le pattern fonctionne

Le header déclare struct Device sans définir ses champs. L'implémentation connaît la structure réelle et fournit des fonctions device_XXX.

Quand l'utiliser

Bibliothèques C, ABI stable, drivers, SDK, composants dont l'implémentation doit rester privée.

Points de vigilance

Chaque opération devient un appel de fonction ; il faut définir clairement ownership, thread-safety et invalidation du handle.

Transposition en C

C'est l'un des idiomes fondamentaux pour obtenir encapsulation et ABI propre en C.

Transposition en C++

En C++, les membres private suffisent souvent ; Pimpl va plus loin lorsque la stabilité ABI/temps de compilation compte.

Lecture de l'exemple

Exemple : le client voit uniquement Device* et device_send(); il ne peut ni connaître ni dépendre des champs internes.

Intention

Cacher la représentation interne d'un type C derrière une déclaration incomplète.

Exemple concret

Une bibliothèque Device expose device_open/send/close mais ses champs privés restent dans device.c.

Transposition

C : typedef struct Device Device; dans le header.

C++ : Equivalent conceptuel : Pimpl ou classe avec champs privés.

Mini-exemple C

```
/* device.h */
typedef struct Device Device;
Device *device_open(const char *name);
int device_send(Device *, const void *, size_t);
void device_close(Device *);
```

Mini-exemple C++

```
class Device { /* champs privés ou pimpl */ };
```

Pimpl

Idée en une phrase

Déplacer les détails privés d'une classe C++ dans un objet d'implémentation alloué séparément.

Problème résolu

Le header inclut de nombreux types privés, ralentit la compilation et rend l'ABI fragile : ajouter un champ change sizeof(Widget).

Comment le pattern fonctionne

La classe publique contient uniquement std::unique_ptr<Impl>. Impl est défini dans le .cpp avec les dépendances lourdes.

Quand l'utiliser

Bibliothèques binaires, gros projets, API stables, classes dépendant de frameworks lourds.

Points de vigilance

Pimpl ajoute une allocation/indirection et complique légèrement move/copy/destructeur. Ne l'utilisez pas pour chaque petite classe interne.

Transposition en C

Le concept correspond en pratique à Opaque Pointer, déjà naturel en C.

Transposition en C++

Le destructeur de la classe publique est généralement défini dans le .cpp après la définition de Impl afin que unique_ptr puisse détruire le type complet.

Lecture de l'exemple

Exemple : Widget.hpp ne dépend plus du moteur graphique ; seul Widget::Impl dans widget.cpp connaît ces headers.

Intention

Cacher l'implémentation d'une classe C++ dans un type Impl défini dans le .cpp afin de réduire dépendances et stabiliser l'ABI.

Exemple concret

Widget.hpp n'inclut ni OpenGL ni les gros headers internes; seul Widget.cpp connaît ces détails.

Transposition

C : En C, opaque pointer offre le même objectif.

C++ : unique_ptr<Impl> comme seul membre privé public-visible.

Mini-exemple C++

```
// widget.hpp
class Widget {
    struct Impl;
    std::unique_ptr<Impl> p_;
public:
    Widget(); ~Widget();
    void draw();
};
```

Handle + génération

Idée en une phrase

Référencer un objet par un index compact accompagné d'une génération pour détecter les références périmées.

Problème résolu

Les pointeurs bruts vers des objets stockés dans des pools deviennent dangling lorsque l'emplacement est réutilisé.

Comment le pattern fonctionne

Un handle contient index + generation. À la libération, la génération du slot est incrémentée. Une ancienne référence échoue alors au test de validité.

Quand l'utiliser

ECS, moteurs de jeu, resource managers, pools d'objets, handles GPU.

Points de vigilance

Gérez le wrap-around des générations et les accès concurrents. Le handle ne garantit pas à lui seul la synchronisation de l'objet.

Transposition en C

En C, un struct de deux uint32_t est simple, sérialisable et très efficace.

Transposition en C++

En C++, un type fort EntityId évite de mélanger les handles de familles différentes.

Lecture de l'exemple

Exemple : EntityId{42,7} devient invalide si le slot 42 est libéré puis recréé avec génération 8.

Intention

Référencer une ressource par {index,génération} pour détecter les handles périmés après réutilisation d'un slot.

Exemple concret

Un moteur de jeu détruit Entity 42 puis réutilise le slot 42; l'ancienne référence doit devenir invalide.

Transposition

C : Struct {uint32 index,génération} vérifiée dans le manager.
C++ : Même idée avec type fort et arrays de generations.

Mini-exemple C

```
typedef struct { uint32_t index, generation; } EntityId;
bool entity_valid(World *w, EntityId id) {
    return id.index < w->cap && w->generation[id.index] == id.generation;
}
```

Mini-exemple C++

```
struct EntityId { uint32_t index, generation; };
bool valid(EntityId id) const { return generations_[id.index] == id.generation; }
```

Type Erasure

Idée en une phrase

Manipuler des objets de types différents derrière une même interface de valeur sans exposer leur type concret.

Problème résolu

On veut stocker des callables ou objets hétérogènes sans imposer une hiérarchie d'héritage commune.

Comment le pattern fonctionne

Un wrapper stocke un void*/buffer et un ensemble d'opérations effacées (call, destroy, copy...). Le type concret est connu uniquement lors de la construction du wrapper.

Quand l'utiliser

std::function, any task, callbacks, plugin local, polymorphisme de valeur.

Points de vigilance

Le type erasure ajoute indirection et parfois allocation. Le contrat doit être soigneusement défini car les capacités absentes ne sont plus visibles statiquement.

Transposition en C

En C, le couple void *obj + pointeurs de fonctions est déjà une forme explicite de type erasure.

Transposition en C++

std::function, std::any et certains wrappers custom emploient ce principe ; la SBO peut éviter une allocation pour les petits objets.

Lecture de l'exemple

Exemple : AnyTask peut contenir un job réseau, un calcul ou une lambda C++ et expose seulement run/destroy.

Intention

Manipuler des types hétérogènes derrière une interface de valeur sans imposer une classe de base commune.

Exemple concret

Un Scheduler stocke différentes tâches callable dans le même conteneur.

Transposition

C : Manuellement : void* + table de fonctions copy/destroy/call.

C++ : std::function, std::any ou wrapper custom.

Mini-exemple C

```
typedef struct { void *obj; void (*call)(void*); void (*destroy)(void*); } AnyTask;
void task_run(AnyTask *t){ t->call(t->obj); }
```

Mini-exemple C++

```
std::vector<std::function<void()>> jobs;
jobs.push_back([id]{ process(id); });
```

C RTP

Idée en une phrase

Utiliser l'héritage de template pour obtenir du polymorphisme statique à la compilation.

Problème résolu

On souhaite factoriser un comportement générique sans coût de virtual dispatch et en laissant la classe dérivée fournir certaines opérations.

Comment le pattern fonctionne

Base<Derived> caste *this vers Derived& et appelle des méthodes de Derived. Le compilateur connaît le type exact et peut inline.

Quand l'utiliser

Bibliothèques numériques, mixins, interfaces statiques, opérateurs génériques, code haute performance.

Points de vigilance

C RTP augmente la complexité des messages d'erreur et couple fortement Base à l'API attendue du Derived. Les concepts C++20 sont souvent utiles pour documenter ces exigences.

Transposition en C

Pas d'équivalent direct en C : le choix est plutôt macros, génération de code ou fonctions génériques explicites.

Transposition en C++

C'est un idiome C++ compile-time ; il ne remplace pas le polymorphisme dynamique lorsqu'on doit stocker des types hétérogènes derrière une même interface runtime.

Lecture de l'exemple

Exemple : Printable<Invoice>::print() appelle Invoice::print_impl() sans méthode virtuelle.

Intention

Utiliser l'héritage template avec le type dérivé pour obtenir polymorphisme statique et mixins sans virtual.

Exemple concret

Printable<Invoice> fournit print() qui appelle Invoice::print_impl().

Transposition

C : Pas d'équivalent direct générique; macros ou composition peuvent approcher certains usages.
C++ : template<class Derived> Base<Derived>.

Mini-exemple C++

```
template<class D>
struct Printable {
    void print() const { static_cast<const D&>(*this).print_impl(); }
};
struct Invoice : Printable<Invoice> {
    void print_impl() const { std::cout << "invoice"; }
};
```

Policy-Based Design

Idée en une phrase

Assembler un type à partir de politiques orthogonales paramétrables, souvent à la compilation.

Problème résolu

Plusieurs dimensions de comportement doivent être combinées sans créer une hiérarchie de classes pour chaque combinaison.

Comment le pattern fonctionne

Le composant principal délègue des décisions à des politiques : Eviction, Locking, Allocation, ErrorPolicy, etc. Les politiques peuvent être des templates, types ou fonctions.

Quand l'utiliser

Caches, conteneurs, bibliothèques système, allocation, synchronisation, calcul numérique.

Points de vigilance

Trop de politiques rendent les types difficiles à lire et explosent le nombre d'instanciations template. Réservez-les aux axes réellement indépendants et performants.

Transposition en C

En C, une structure de callbacks/ctx peut représenter les politiques au runtime.

Transposition en C++

En C++, templates de classe et héritage privé/EBO permettent des politiques zero-overhead.

Lecture de l'exemple

Exemple : Cache<LruEviction, NoLock> et Cache<FifoEviction, MutexLock> partagent le même coeur mais des politiques différentes.

Intention

Paramétrer le comportement d'un composant par des types policy choisis à la compilation.

Exemple concret

Cache<Key, LruEviction, MutexLock> peut changer politique d'éviction ou synchronisation sans coût virtual.

Transposition

C : Peut se simuler par struct de callbacks passée à l'instance.
C++ : Templates avec politiques.

Mini-exemple C

```
typedef struct { int (*victim)(void *ctx); void *ctx; } EvictionPolicy;
typedef struct { EvictionPolicy eviction; /* ... */ } Cache;
```

Mini-exemple C++

```
template<class Eviction, class Locking>
class Cache : private Eviction, private Locking { /* ... */ };
using FastCache = Cache<LruEviction, NoLock>;
```

Intrusive Containers

Idée en une phrase

Stocker les liens de la structure de données directement dans l'objet au lieu d'allouer un noeud séparé.

Problème résolu

Les conteneurs classiques imposent une allocation par noeud ou une indirection supplémentaire, coûteuse en systèmes contraints.

Comment le pattern fonctionne

L'objet contient next/prev ou un hook. Le conteneur manipule ces liens mais ne possède pas forcément l'objet.

Quand l'utiliser

Kernels, schedulers, moteurs de jeu, systèmes temps réel, listes de ressources déjà gérées ailleurs.

Points de vigilance

L'ownership devient plus subtil et un objet ne peut pas être simultanément dans deux listes utilisant le même hook. Les invariants doivent être rigoureux.

Transposition en C

En C, les linked lists intrusives sont extrêmement courantes : struct Task contient Task *next.

Transposition en C++

Boost.Intrusive fournit des conteneurs robustes avec hooks multiples et zéro allocation de noeud.

Lecture de l'exemple

Exemple : le scheduler place Task directement dans ready list sans créer un wrapper ListNode.

Intention

Stocker les liens de la structure de données directement dans l'objet pour éviter une allocation de nœud séparée.

Exemple concret

Un ordonnanceur temps réel place Task dans plusieurs listes sans allocation heap.

Transposition

C : Task contient next/prev.
C++ : boost::intrusive ou structure custom.

Mini-exemple C

```
typedef struct Task { struct Task *next; int priority; } Task;
void push(Task **head, Task *t){ t->next=*head; *head=t; }
```

Mini-exemple C++

```
struct Task : boost::intrusive::list_base_hook<> { int priority; };
boost::intrusive::list<Task> ready;
```

Point de vigilance - La durée de vie des objets doit être strictement maîtrisée : le conteneur ne possède généralement pas les éléments.

Copy-on-Write

Idée en une phrase

Partager les données tant qu'elles sont en lecture seule et effectuer une copie uniquement au moment d'une modification.

Problème résolu

Copier immédiatement de gros buffers est coûteux alors que la plupart des copies logiques ne seront jamais modifiées.

Comment le pattern fonctionne

Plusieurs objets partagent le même buffer avec un compteur de références. Avant écriture, detach() clone si le buffer n'est pas unique.

Quand l'utiliser

Snapshots, images, gros documents, structures persistantes - uniquement si les lectures dominent réellement.

Points de vigilance

COW complique fortement la concurrence et les garanties de constness. Les standard strings modernes n'utilisent plus COW pour ces raisons.

Transposition en C

En C, un header avec refs et clone à l'écriture permet une implémentation explicite.

Transposition en C++

std::shared_ptr + unique() peut servir de base, mais la synchronisation et les coûts atomiques doivent être mesurés.

Lecture de l'exemple

Exemple : deux Image partagent Buffer ; le premier setPixel() d'une copie force la duplication avant mutation.

Intention

Partager un état tant qu'il est en lecture et effectuer une copie seulement lors de la première modification.

Exemple concret

Deux images partagent un gros buffer; l'une ne copie les pixels que lorsqu'elle est modifiée.

Transposition

C : Refcount manuel + ensure_unique avant écriture.
C++ : shared_ptr + detach/clone avant mutation.

Mini-exemple C

```
typedef struct { int refs; size_t n; unsigned char data[]; } Buffer;
void image_detach(Image *img) {
    if (img->buf->refs>1) img->buf = buffer_clone(img->buf);
}
```

Mini-exemple C++

```
void Image::setPixel(...) {
    if (!buf_.unique()) buf_ = std::make_shared<Buffer>(*buf_);
    /* mutate */
}
```

Point de vigilance - La concurrence et les coûts de copie cachés peuvent rendre COW surprenant; à utiliser avec intention claire.

Small Buffer Optimization

Idée en une phrase

Stocker les petites valeurs directement dans l'objet et n'allouer sur le heap qu'au-delà d'un seuil.

Problème résolu

De très nombreuses petites chaînes, vecteurs ou callables paient le coût d'une allocation dynamique pourtant évitable.

Comment le pattern fonctionne

L'objet contient un buffer inline et un indicateur. Tant que la taille tient dans ce buffer, aucun malloc/new n'est nécessaire ; sinon il bascule vers le heap.

Quand l'utiliser

SmallString, SmallVector, std::function-like wrappers, messages courts, structures haute performance.

Points de vigilance

SBO augmente sizeof(objet), complexifie move/copy et peut être contre-productive si la majorité des objets dépassent le seuil.

Transposition en C

En C, une union small[] / pointer et un flag heap suffisent.

Transposition en C++

En C++, de nombreuses bibliothèques proposent small_vector ; le type erasure et les chaînes custom emploient aussi ce principe.

Lecture de l'exemple

Exemple : SmallString garde jusqu'à 23 caractères inline ; une URL longue alloue seulement lorsqu'elle dépasse cette capacité.

Intention

Stocker les petites valeurs directement dans l'objet et utiliser le heap uniquement au-delà d'un seuil.

Exemple concret

Une SmallString garde jusqu'à 23 caractères inline et évite malloc pour les noms courts.

Transposition

C : Union {inline buffer / heap pointer} + tag/length.

C++ : Technique utilisée dans small_vector/small_string et parfois std::function.

Mini-exemple C

```
typedef struct {
    size_t len;
    bool heap;
    union { char small[24]; char *ptr; } data;
} SmallString;
```

Mini-exemple C++

```
class SmallVec {
    std::array<T,8> local_;
    std::unique_ptr<T[]> heap_;
    // bascule au-delà de 8 éléments
};
```

Object Handle / Resource Manager

Idée en une phrase

Centraliser la possession et la résolution des ressources, tandis que les clients manipulent des identifiants légers.

Problème résolu

Passer des pointeurs directs vers textures, sons ou fichiers rend la durée de vie difficile à coordonner et empêche facilement rechargement, cache ou éviction.

Comment le pattern fonctionne

Le ResourceManager possède les ressources. Un handle/ID identifie une entrée ; get/resolve transforme l'ID en objet courant.

Quand l'utiliser

Moteurs de jeu, assets, GPU resources, caches partagés, gros objets chargés/déchargés dynamiquement.

Points de vigilance

La résolution ajoute un lookup ; un handle périmé doit être détecté (souvent via génération). Définissez clairement ce qui se passe lors d'une éviction.

Transposition en C

En C, un uint32_t ou struct handle indexe une table du ResourceManager.

Transposition en C++

En C++, utilisez un type fort TextureId plutôt qu'un entier brut et exposez Texture& ou un view contrôlé.

Lecture de l'exemple

Exemple : les entités conservent TextureId ; le manager peut recharger la texture sans changer toutes les références clientes.

Intention

Découpler l'identité logique d'une ressource de son adresse mémoire réelle.

Exemple concret

Un jeu manipule TextureId; le ResourceManager peut recharger la texture GPU sans changer l'ID vu par les systèmes.

Transposition

C : ID entier + table manager.
C++ : Type fort Handle<T> + manager/cache.

Mini-exemple C

```
typedef uint32_t TextureId;
Texture *texture_resolve(ResourceManager *rm, TextureId id);
```

Mini-exemple C++

```
struct TextureId { uint32_t value; };
Texture& ResourceManager::get(TextureId id) { return textures_.at(id.value); }
```

8. Concurrency et systèmes

Producer-Consumer

Idée en une phrase

Découpler les producteurs de travail des consommateurs au moyen d'une file partagée.

Problème résolu

Le rythme de production et le rythme de traitement diffèrent ; exécuter directement dans le thread producteur crée du couplage temporel.

Comment le pattern fonctionne

Les producteurs poussent des éléments dans une queue. Un ou plusieurs consommateurs attendent, retirent et traitent les éléments. La queue définit la politique de backpressure.

Quand l'utiliser

Logging, traitement vidéo, réseau, pipelines, jobs, acquisition capteur, streaming.

Points de vigilance

Queue bornée ou non, arrêt, fairness, ordre et comportement lorsque la queue est pleine sont des décisions essentielles.

Transposition en C

En C, mutex + condition variable autour d'un ring/list queue constitue une implémentation classique.

Transposition en C++

En C++, une BlockingQueue<T> encapsule synchronisation et peut exposer close()/pop().

Lecture de l'exemple

Exemple : le thread réseau produit des Job ; des workers consommateurs les traitent indépendamment.

Intention

Découpler producteurs et consommateurs par une file partagée avec politique de backpressure.

Exemple concret

Des threads réseau produisent des Jobs; des workers les traitent sans bloquer l'acceptation de nouvelles connexions.

Transposition

C : Queue protégée mutex/condvar ou lock-free selon besoin.
C++ : blocking_queue + condition_variable, channels ou concurrent queue.

Mini-exemple C

```
void producer(Queue *q, Job j){ queue_push(q,j); cond_signal(&q->not_empty); }
Job consumer(Queue *q){ return queue_wait_pop(q); }
```

Mini-exemple C++

```
BlockingQueue<Job> q;
// producer: q.push(job)
// consumer: while(auto j=q.pop()) process(*j);
```

Thread Pool

Idée en une phrase

Réutiliser un nombre borné de threads pour exécuter de nombreuses tâches courtes.

Problème résolu

Créer un thread par tâche coûte cher, peut épuiser le système et rend la concurrence incontrôlée.

Comment le pattern fonctionne

Une file de jobs alimente N workers permanents. submit() ajoute une tâche ; un worker disponible la retire et l'exécute.

Quand l'utiliser

Serveurs, calcul parallèle, traitements asynchrones, tâches CPU/I/O contrôlées.

Points de vigilance

Le dimensionnement dépend du type de charge. Attention aux deadlocks si des jobs du pool attendent d'autres jobs du même pool ; gérez shutdown proprement.

Transposition en C

En C, le pool contient threads + queue synchronisée + flag stop.

Transposition en C++

En C++, std::jthread facilite l'arrêt ; futures/promises peuvent transporter les résultats.

Lecture de l'exemple

Exemple : submit([id]{ compute(id); }) n'alloue pas un nouveau thread : un worker existant prend la tâche.

Intention

Réutiliser un nombre borné de threads pour exécuter des tâches, au lieu de créer un thread par tâche.

Exemple concret

Un serveur possède 8 workers qui consomment une file de requêtes CPU-bound.

Transposition

C : Threads créés au démarrage + file de jobs + condition variable.

C++ : vector<jthread> + queue<function<void()>>.

Mini-exemple C

```
typedef struct { Thread workers[8]; Queue jobs; } ThreadPool;
void submit(ThreadPool *p, Job j){ queue_push(&p->jobs,j); }
```

Mini-exemple C++

```
pool.submit([id]{ compute(id); }); // worker déjà existant exécute la tâche
```

Point de vigilance - Un Thread Pool contrôle les threads; un Object Pool réutilise des objets. Ils peuvent coexister mais ne résolvent pas le même problème.

Active Object

Idée en une phrase

Associer un objet à son propre contexte d'exécution et transformer ses appels en messages mis en file.

Problème résolu

Plusieurs threads appellent un composant non thread-safe. Verrouiller chaque méthode mélange synchronisation et logique métier.

Comment le pattern fonctionne

Les méthodes publiques enfilent des requêtes. Un worker unique les exécute séquentiellement sur l'état interne de l'objet.

Quand l'utiliser

Drivers série, acteurs simples, composants matériels, services avec affinité de thread, UI backend.

Points de vigilance

Les résultats deviennent souvent asynchrones. Il faut gérer backlog, arrêt, erreurs et appels qui ne doivent pas bloquer le worker.

Transposition en C

En C, une queue + worker thread dédié matérialise le pattern.

Transposition en C++

En C++, std::jthread et une BlockingQueue<Message> permettent une implémentation concise avec RAII.

Lecture de l'exemple

Exemple : ActiveSerial::send() ne touche jamais directement au port ; le worker sérialise tous les write().

Intention

Séparer l'appel d'une opération de son exécution en donnant à l'objet sa propre file et son contexte d'exécution.

Exemple concret

Un SerialPort accepte send() depuis plusieurs threads mais sérialise réellement les écritures dans son worker interne.

Transposition

C : Struct + queue de messages + thread associé.
C++ : Objet contenant queue, mutex, condvar et jthread.

Mini-exemple C

```
void serial_send(ActiveSerial *s, Message m) { queue_push(&s->q,m); }
void *serial_worker(void *arg) { for(;;) write_msg(queue_pop(&((ActiveSerial*)arg)->q)); }
```

Mini-exemple C++

```
class ActiveSerial {
    BlockingQueue<Message> q_;
    std::jthread worker_{[this]{ while(auto m=q_.pop()) write(*m); }};
public: void send(Message m){ q_.push(std::move(m)); }
};
```

Reactor

Idée en une phrase

Attendre des événements de disponibilité puis dispatcher les handlers correspondants.

Problème résolu

Des milliers de sockets seraient trop coûteuses avec un thread bloquant par connexion.

Comment le pattern fonctionne

Un event demultiplexer (select/poll/epoll/kqueue) indique quels handles sont prêts. Le Reactor appelle alors le callback enregistré pour chacun.

Quand l'utiliser

Serveurs réseau, boucles d'événements, GUI, multiplexage de descripteurs.

Points de vigilance

Le handler ne doit pas bloquer longtemps, sinon toute la boucle est retardée. Le modèle reste basé sur « prêt à faire », pas « opération terminée ».

Transposition en C

En C, epoll_wait + pointeur Handler est une forme très directe.

Transposition en C++

En C++, bibliothèques comme Asio encapsulent ce pattern avec handlers et executors.

Lecture de l'exemple

Exemple : epoll signale fd lisible ; reactor appelle connection.readAvailable() pour consommer ce qui est prêt.

Intention

Attendre des événements de disponibilité puis dispatcher vers des handlers enregistrés.

Exemple concret

Un serveur Linux attend EPOLLIN/EPOLLOUT pour des milliers de sockets et appelle le handler concerné.

Transposition

C : select/poll/epoll/kqueue + table de handlers.
C++ : Même modèle via wrappers de boucle d'événements.

Mini-exemple C

```
for (;;) {
    int n=epoll_wait(epfd, events, MAX_EVENTS, -1);
    for(int i=0;i<n;i++) {
        Handler *h=events[i].data.ptr;
        h->on_ready(h, events[i].events);
    }
}
```

Mini-exemple C++

```
reactor.onReadable(fd, [&]{ connection.readAvailable(); });
```

Proactor

Idée en une phrase

Lancer une opération asynchrone et être notifié lorsqu'elle est réellement terminée.

Problème résolu

Le Reactor indique seulement qu'une opération est possible ; certaines plateformes permettent au kernel/runtime d'effectuer l'I/O elle-même.

Comment le pattern fonctionne

Le programme soumet `async_read`. Le système effectue le transfert puis appelle un completion handler ou reprend une coroutine avec le résultat.

Quand l'utiliser

IOCP Windows, `io_uring` selon usage, bibliothèques async, opérations de fichiers/réseau asynchrones.

Points de vigilance

La durée de vie des buffers et contextes doit couvrir l'opération. Les erreurs arrivent à la complétion et non au moment du submit.

Transposition en C

En C, `callback + ctx` est la forme classique de completion handler.

Transposition en C++

En C++, `co_await` permet d'écrire le flux de manière séquentielle tout en conservant le modèle de complétion asynchrone.

Lecture de l'exemple

Exemple : `async_read()` est soumis ; `on_read_complete` reçoit `n` et `err` lorsque l'I/O est terminée.

Intention

Lancer une opération asynchrone et traiter sa complétion lorsque le système signale qu'elle est terminée.

Exemple concret

Sous Windows IOCP, on lance une lecture overlapped puis le completion port signale la fin de l'I/O.

Transposition

C : API async plateforme + `callback/completion queue`.

C++ : `Future/coroutine/callback` ou `IOCP/ASIO completion handlers`.

Mini-exemple C

```
async_read(fd, buffer, on_read_complete, ctx);
void on_read_complete(void *ctx, int n, int err) {
    if(!err) process(ctx, n);
}
```

Mini-exemple C++

```
co_await socket.async_read_some(buffer); // la continuation reprend à la complétion
```

Point de vigilance - Le Reactor signale généralement « prêt à faire » ; le Proactor signale « opération terminée ».

Half-Sync / Half-Async

Idée en une phrase

Séparer une couche asynchrone de gestion d'événements d'une couche synchrone de traitement métier, reliées par une queue.

Problème résolu

Le réseau doit rester non bloquant mais le traitement métier est plus simple à écrire en mode synchrone ou peut être coûteux.

Comment le pattern fonctionne

L'évent loop reçoit et normalise les requêtes puis les place dans une queue. Un pool de workers traite ces requêtes avec un modèle bloquant classique.

Quand l'utiliser

Serveurs historiques et hybrides, proxies, traitements réseau + CPU.

Points de vigilance

La queue est une frontière de backpressure. Si les workers sont saturés, l'évent loop peut continuer à accepter du travail trop vite.

Transposition en C

En C, `callback réseau push` dans `work queue` ; worker bloque sur `pop`.

Transposition en C++

En C++, un Reactor peut soumettre les traitements longs à un `ThreadPool`.

Lecture de l'exemple

Exemple : `on_socket_ready` reste très court ; `handle(request)` s'exécute ensuite dans le pool synchrone.

Intention

Séparer une couche asynchrone d'I/O d'une couche synchrone de traitement au moyen d'une file.

Exemple concret

Une boucle epoll reçoit les événements réseau et dépose les requêtes dans un pool de workers synchrones.

Transposition

C : Reactor thread -> queue -> worker threads.
C++ : Event loop + blocking queue + thread pool.

Mini-exemple C

```
// async half
on_socket_ready(Request r) { queue_push(&work, r); }
// sync half
worker() { for(;;) handle(queue_pop(&work)); }
```

Mini-exemple C++

```
eventLoop.onRequest([&](Request r){ workers.submit([r]{ handle(r); }); });
```

Double Buffer

Idée en une phrase

Maintenir deux buffers pour que l'un soit consommé pendant que l'autre est préparé.

Problème résolu

Lecture et écriture concurrentes du même état provoquent tearing, incohérences ou nécessitent un verrou autour de toute la production.

Comment le pattern fonctionne

front est stable pour le consommateur ; back est modifié par le producteur. Une fois prêt, on échange les rôles en une opération contrôlée.

Quand l'utiliser

Rendu graphique, audio, simulation, snapshots de configuration, télémétrie.

Points de vigilance

Le swap doit être correctement synchronisé entre threads. Deux buffers doublent la mémoire et ne résolvent pas tous les problèmes de fréquence producteur/consommateur.

Transposition en C

En C, deux pointeurs et un swap suffisent pour le modèle simple.

Transposition en C++

En C++, std::swap rend l'intention explicite ; pour le multithread, on peut combiner index atomique ou verrou court.

Lecture de l'exemple

Exemple : render(back), swap(front, back), puis display(front) garantit que l'affichage voit une frame complète.

Intention

Alterner deux buffers : l'un est lu pendant que l'autre est écrit, puis on les échange.

Exemple concret

Un moteur graphique construit l'image suivante dans backBuffer pendant que frontBuffer est affiché.

Transposition

C : Deux buffers + swap de pointeurs/indices.
C++ : Deux containers/frames + std::swap.

Mini-exemple C

```
Frame *front=&frames[0], *back=&frames[1];
render(back);
Frame *tmp=front; front=back; back=tmp;
display(front);
```

Mini-exemple C++

```
render(back);
std::swap(front, back);
display(front);
```

Ring Buffer

Idée en une phrase

Utiliser un tableau circulaire à capacité fixe pour obtenir une queue très prévisible et sans déplacement des éléments.

Problème résolu

Il faut un FIFO performant, borné et peu allouant, par exemple pour audio, réseau ou communication entre threads.

Comment le pattern fonctionne

head indique où écrire, tail où lire ; les indices reviennent au début avec modulo/masque. L'état full/empty doit être distingué.

Quand l'utiliser

Audio temps réel, UART, paquets, logs, SPSC queues, capteurs.

Points de vigilance

Définissez précisément la politique en cas de plein : refuser, écraser l'ancien, bloquer. Les versions lock-free exigent une discipline mémoire correcte.

Transposition en C

En C, un tableau fixe et deux indices donnent une implémentation compacte.

Transposition en C++

En C++, un template RingBuffer<T,N> peut encapsuler les invariants ; les variantes SPSC atomiques sont très courantes.

Lecture de l'exemple

Exemple : audio.try_push(sample) ajoute sans allocation ; try_pop() lit dans l'ordre FIFO avec latence prévisible.

Intention

Utiliser un buffer circulaire borné pour streaming et produire-consumer avec excellente localité mémoire.

Exemple concret

Un périphérique audio écrit des samples tandis qu'un thread audio les lit en continu.

Transposition

C : Array fixe + head/tail modulo capacité.

C++ : std::array + indices atomiques ou ring buffer library.

Mini-exemple C

```
bool rb_push(Ring *r, Sample x){
    size_t n=(r->head+1)%CAP; if(n==r->tail) return false;
    r->data[r->head]=x; r->head=n; return true;
}
bool rb_pop(Ring *r, Sample *x){
    if(r->tail==r->head) return false;
    *x=r->data[r->tail]; r->tail=(r->tail+1)%CAP; return true;
}
```

Mini-exemple C++

```
RingBuffer<Sample,4096> audio;
audio.try_push(sample); auto s=audio.try_pop();
```

9. Boussoles de choix

Quel pattern pour quel problème ?

Problème observé	Pattern(s) à examiner	Question discriminante
Création varie	Factory Method / Abstract Factory	Un produit ou une famille de produits ?
Construction complexe	Builder	Plusieurs étapes/options rendent-elles le constructeur illisible ?
API incompatible	Adapter	Faut-il traduire un contrat externe vers le vocabulaire local ?
Options combinables	Decorator	Les responsabilités doivent-elles s'empiler dynamiquement ?
Accès contrôlé/lazy	Proxy	Le contrat reste-t-il le même mais l'accès doit-il être contrôlé ?
Algorithme variable	Strategy	La politique est-elle choisie indépendamment de l'état interne ?
Cycle de vie variable	State	Le comportement change-t-il avec l'état et ses

Problème observé	Pattern(s) à examiner	Question discriminante
		transitions ?
Réactions multiples	Observer	Les abonnés peuvent-ils évoluer indépendamment ?
Action à stocker/rejouer	Command	La requête elle-même a-t-elle un cycle de vie ?
Objets coûteux réutilisables	Object Pool	La réutilisation mesurée évite-t-elle un vrai coût ?
Allocations temporaires groupées	Arena	Peut-on libérer toute la région d'un coup ?
I/O readiness	Reactor	Veut-on être notifié quand une opération peut démarrer ?
I/O completion	Proactor	Veut-on être notifié quand l'opération est terminée ?

Object Pool vs Memory Pool vs Arena vs Flyweight

Mécanisme	Ce qui est réutilisé/partagé	Exemple
Object Pool	Instances construites au cours du temps	Connexions, buffers, projectiles
Memory Pool	Blocs de stockage	Petites allocations fixes en embedded
Arena	Une région entière puis reset global	AST, requête, frame
Flyweight	Etat intrinsèque partagé entre objets simultanés	Styles de glyphes, metadata communes

Checklist avant d'introduire un pattern

- Quel problème concret et récurrent cherche-t-on à résoudre ?
- Quel axe de variation est réellement observé ou très probable ?
- Une fonction, un enum, une structure ou une composition directe suffiraient-ils ?
- Quel couplage le pattern réduit-il réellement ?
- Quel coût cognitif, mémoire ou runtime ajoute-t-il ?
- Le code devient-il plus facile à tester et à expliquer ?
- Les durées de vie et responsabilités sont-elles plus claires ?
- Peut-on retirer le pattern facilement si la variation disparaît ?

Réflexe final : nommer d'abord le problème et l'axe de variation. Si une fonction, une structure simple, un enum ou une composition directe suffit, le pattern plus élaboré peut attendre. En C comme en C++, les patterns sont des outils de conception, pas une checklist à appliquer systématiquement.